The StarForth Cookbook

Patterns, Recipes, and Idioms for FORTH Programmers

Robert A. James

Contents

Part I

FORTH Fundamentals

Chapter 1

Stack Idioms

Chapter 2

Defining Words and Vocabularies

Chapter 3

Factoring Recipes

Part II

Working with Blocks

Chapter 4

Block File Layout

Chapter 5

Block I/O Recipes

Chapter 6

Capsule Programming

6.1 Mama Forth: The Multi-VM Supervisor

Mama Forth is the governing FORTH VM that orchestrates a multi-VM ecosystem. It departs from conventional supervisor design in one decisive respect: Mama Forth is itself a FORTH VM, a first-class citizen carrying the same physics instrumentation as every VM it governs. The supervisor does not stand outside the runtime model; it participates in it.

The ecosystem forms a tree. Mama Forth sits at the root. Beneath it, child VMs each carry their own physics state. Beneath each child, block devices carry per-device physics. Mama observes all three layers and makes system-level decisions: when to spawn or kill a child VM, how to allocate block devices to children, how to enforce governance across the ecosystem, and how to respond to emergencies such as thermal runaway or memory pressure.

The central design insight is that physics metadata becomes a three-tier hierarchy:

- **Word level** — per-word metrics: entropy, temperature, latency.
- **VM level** — per-VM aggregates: child health, resource usage.
- **Block level** — per-block device metrics: I/O patterns, thermal distribution.

6.1.1 VM-Level Physics Metadata

Today's physics model tracks per-word metrics attached to each `DictEntry`:

```
typedef struct {
    uint16_t temperature_q8;   // Thermal state [0x0000, 0xFFFF]
    uint64_t last_active_ns;   // Last execution timestamp
    uint32_t entropy;          // Execution randomness
    int16_t  entropy_slope;    // Rate of entropy change
    uint32_t avg_latency_ns;   // P50 latency
    /* ... */
} DictPhysics;
```

The proposed extension gives each VM its own aggregate physics record. It rolls up thermal state across all words (warmest word, average temperature, count of hot words above `0x8000`), execution load (cumulative executions, words-per-second rate, recency), memory pressure (dictionary heap usage against capacity), reliability (error count, error rate, recovery count), child VM health, block-device affinity, and lifecycle state (uptime, age, run state, criticality class).

```
typedef struct {
    uint16_t vm_temperature_q8;       // Warmest word temperature
    uint16_t vm_avg_temperature_q8;   // Average across all words
    uint32_t vm_hot_word_count;       // Words with temp > 0x8000
```

```
5     uint64_t total_word_executions;
6     uint64_t execution_rate_per_sec;
7     float    dictionary_pressure_pct; // used / max
8     uint32_t error_rate_per_sec;
9     uint32_t active_child_vm_count;
10    uint8_t  vm_state;                // RUNNING, PAUSED, DRAINING,
          DEAD
11    uint8_t  vm_criticality;          // system, critical, normal,
          background
12    /* ... */
13 } VMPhysics;
```

The record attaches directly to the `VM` struct. Every physics event maintains it: `physics_metadata_touch()` updates the word, then resolves the current VM and updates the VM-level aggregates, recomputing running averages and rates.

6.1.2 Block Device Physics Metadata

Block storage (1024 blocks of 1024 bytes) is managed by `block_subsystem.c` (logical-to-physical mapping) and `blkio_factory.c` (pluggable RAM, file, and L4Re backends). Today there is no observability beyond capacity tracking. The proposal attaches a `BlockPhysics` record to each block, tracking thermal state (access frequency), access patterns (read and write counts, P50 and P99 latencies), ownership (owning VM and word), content-type hints, and health (CRC32 checksum, dirty and corruption flags, error count). A device-level aggregate summarizes total reads and writes, hottest and coldest block temperatures, and fragmentation percentage.

Block accessors maintain these metrics inline. On read, the subsystem times the I/O, updates the block's read count and exponential moving-average latency, recomputes temperature from read frequency, and verifies the CRC32 to detect corruption. On write, it does the symmetric work and refreshes the stored checksum.

6.1.3 Mama's Responsibilities

Mama Forth runs with elevated privileges along three axes:

- **Observe** — all word executions in children, all block-device I/O, all child lifecycle events, all errors and anomalies.
- **Control** — spawn and kill child VMs, allocate blocks to children, enforce governance policies, redirect critical work, scale up and down with load.
- **Coordinate** — share block resources, prevent deadlock, balance load across children, degrade gracefully under stress.

Mama is itself instrumented with a `MamaPhysics` record covering its own supervision load, child-management counts, block-management activity, emergency events (thermal warnings, memory-pressure events, orphan recoveries), and governance metrics (violations detected, enforcement actions, audit events).

6.1.4 The Supervision Loop

Mama runs a periodic control loop. Each cycle observes every child's VM-level physics, flags thermal warnings, memory pressure above 90%, and error surges, then inspects every block device for hot blocks and excessive fragmentation. It then makes load-balancing decisions, enforces governance compliance, and sleeps for an adaptive interval derived from its own decision latency.

```
void mama_supervision_loop(void) {
    while (mama_vm.running) {
        for (int i = 0; i < child_vm_count; i++) {
            VM *child = child_vms[i];
            VMPhysics p = child->physics;
            if (p.vm_temperature_q8 > THERMAL_WARNING)
                mama_handle_hot_child(child);
            if (p.dictionary_pressure_pct > 90)
                mama_handle_memory_pressure(child);
            if (p.error_rate_per_sec > ERROR_THRESHOLD)
                mama_handle_error_surge(child);
        }
        mama_balance_load_across_children();
        mama_check_governance_compliance();
        sf_sleep_ns(mama_vm.physics.decision_latency_ns * 2);
    }
}
```

Mama spawns a new child when average child temperature exceeds `0xD000` and capacity remains, allocating a block range to the newcomer and rebalancing the work queue. It kills a child that is unrecoverably faulted, idle, or under unrecoverable memory pressure — first recovering the orphan's blocks, then tearing the VM down. It rebalances block allocation by grouping each child's hottest blocks together to reduce fragmentation and keep hot data near its consumer.

6.1.5 Three-Tier Feedback Loop

Metrics flow upward. Word execution updates word temperature and latency; the VM aggregates word metrics into VM state; Mama aggregates VM state into system state, which is logged across all three tiers to the analytics heap. Decisions flow downward. A Mama decision ("Child 2 is too hot, spawn Child 3") creates a VM and allocates blocks; the affected child adjusts its batch groups and sampling rate; word-level optimization boosts priority for hot words; and the block device responds by relocating cold blocks, compacting fragmentation, and prefetching predicted accesses.

6.1.6 Worked Example: Thermal Emergency

A concrete sequence illustrates the loop. A child VM runs normally until a `BLOCK_WRITE` word heats to `0xC000` and propagates that temperature to the VM. Mama's next cycle detects the warning together with elevated dictionary pressure and an anomalous error rate of 100 errors per second. Mama responds: it spawns a second child for load shedding, reassigns cold work to it, marks the first child's hot blocks protected, temporarily raises that child's heap, and logs the emergency to governance. Within a few seconds the original child cools and stabilizes; the system keeps the new child because it now carries useful work, retiring it only once the first child cools below the safe threshold.

6.1.7 Implementation Phases

- **Phase 1 — Foundation.** Word-level physics and execution hooks are complete; VM-level aggregation and block-device physics remain.

- **Phase 2 — Single-VM Control.** Word-level scheduling and memory hints are complete; VM-level health monitoring and observe-only Mama supervision remain.

- **Phase 3 — Multi-VM Orchestration.** Child spawn and kill, block reallocation, cross-child load balancing, and emergency response.
- **Phase 4 — Mama Optimization.** Mama's own physics tuning, predictive spawning, guided load balancing, and governance enforcement at scale.

Part III

Physics Runtime Recipes

Chapter 7

Understanding Execution Heat

Chapter 8

Tuning the Rolling Window

Chapter 9

Pipelining and Prefetch

Chapter 10

Reading Heartbeat Output

10.1 Heartbeat CSV Export

The heartbeat subsystem emits real-time multivariate VM metrics to `stderr` in CSV form, exposing StarForth's adaptive runtime to external analysis. Each row carries eleven performance indicators sampled once per heartbeat tick — nominally every millisecond — producing a continuous time series of the running system's internal state.

The design favours streaming over buffering. Metrics are read directly from VM state with no intermediate copy, each row is flushed immediately so that downstream tools see data as it is produced, and the stream is confined to `stderr` to keep `stdout` reserved for test output. The emitting thread is a background `pthread` decoupled from the main interpreter, so capture does not perturb execution timing beyond a fixed, small cost per tick.

10.1.1 CSV Format

The VM emits raw rows only; column names are supplied separately by analysis tooling. The reference header is:

```
tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,
    bucket_hits_delta,
word_executions_delta,hot_word_count,avg_word_heat,window_width,
predicted_label_hits,estimated_jitter_ns
```

The *Status* column below reflects which fields carry live values and which currently emit a placeholder zero pending delta-tracking work.

10.1.2 Capturing Metrics

The simplest capture redirects `stderr` to a file and discards `stdout`:

```
# Capture metrics, discard test output
./build/amd64/fastest/starforth --doe 2>heartbeat.csv 1>/dev/null

# Keep both streams, separated
./build/amd64/fastest/starforth --doe 1>test_output.txt 2>heartbeat.
    csv
```

Because rows are flushed as they are produced, the stream can be monitored live — for example piped through `csvlook` from `csvkit`, or followed with `tail -f`. For post-processing, prepend the reference header and load the result into R or pandas:

```
{ echo "tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,\
bucket_hits_delta,word_executions_delta,hot_word_count,avg_word_heat
    ,\
```

Table 10.1: Heartbeat CSV columns

Column	Type	Unit	Description	Status
`tick_number`	`uint32`	count	Monotonic tick counter since VM start	live
`elapsed_ns`	`uint64`	ns	Time since VM initialization	live
`tick_interval_ns`	`uint64`	ns	Time since previous tick (actual vs. nominal 1 ms)	live
`cache_hits_delta`	`uint32`	count	Hotwords cache hits since last tick	TODO
`bucket_hits_delta`	`uint32`	count	Bucket-level cache hits since last tick	TODO
`word_executions_delta`	`uint32`	count	FORTH word executions since last tick	TODO
`hot_word_count`	`uint64`	count	Words at or above the heat promotion threshold (≥ 10)	live
`avg_word_heat`	`double`	heat	Mean execution heat across the dictionary ($\mathbb{Q}_{48.16} \to$ double)	live
`window_width`	`uint32`	entries	Current rolling window effective size (adaptive)	live
`predicted_label_hits`	`uint32`	count	Successful pipelining speculation hits	TODO
`estimated_jitter_ns`	`double`	ns	Absolute deviation from the nominal 1 ms interval	live

```
window_width,predicted_label_hits,estimated_jitter_ns"; \
  cat heartbeat.csv; } > analysis.csv
```

10.1.3 Analysis Use Cases

The eleven-dimensional series supports three broad classes of analysis. For dynamics modeling, the trajectory admits phase-space reconstruction to locate attractor basins in the adaptive parameter space, Lyapunov-exponent estimation to quantify stability, and mutual-information analysis to discover coupling between metrics. For tuning validation, window-width adaptation can be correlated against prediction accuracy, heat-decay behaviour against stale word accumulation, and jitter against thread-scheduling effects. For workload profiling, the same data yields hot-word churn rate, execution density (`word_executions_delta` over `tick_interval_ns`), and cache efficiency relative to working-set size.

10.1.4 Implementation

Capture is performed by `heartbeat_capture_tick_snapshot()`, which reads the monotonic tick counter, computes elapsed and inter-tick time, walks the dictionary to count hot words and their mean heat, samples the rolling window size, and estimates jitter as the absolute difference between the actual and nominal interval. The walk is linear in dictionary size (on the order of a few hundred words). Emission is performed by `heartbeat_emit_tick_row()`, which formats the eleven values and flushes `stderr` immediately. Both are invoked once per cycle from the background heartbeat thread.

The thread runs without the original 50 ms startup delay, so metrics begin flowing immediately even for short-running DoE workloads. The trade-off is that the first few ticks may show transient values during the word-registration phase.

10.1.5 Known Limitations

Three groups of fields are not yet wired to live deltas and currently emit zero: the cache, bucket, and word-execution counters all require per-tick baseline tracking in the heartbeat state. Separately, the source notes an uninitialized `run_start_ns`, which can produce implausibly large `elapsed_ns` values in the first ticks until the field is seeded at VM initialization, and a missing aggregation of per-word prefetch hits behind `predicted_label_hits`.

The runtime cost is small: emission adds well under one percent CPU at the nominal tick rate, the snapshot is a stack-allocated structure of under a hundred bytes, and the output volume is on the order of a hundred kilobytes per second of execution.

Chapter 11

Physics Freeze and Thaw

Part IV

Design of Experiments

Chapter 12

The 2^7 Factorial Design

Chapter 13

Running the Factorial Harness

Chapter 14

Heartbeat DoE

Chapter 15

Optimization DoE

Part V

Security and ACL Recipes

Chapter 16

Word-Level ACL Basics

Chapter 17

The Zuse Superuser Console

Chapter 18

Emergency Console Fallback

Appendix A

Build Flags Quick Reference

A.1 Build Options Reference

StarForth exposes a comprehensive set of build-time configuration options controlling optimizations, feature selection, platform targeting, and debugging. All options are passed as preprocessor defines via `CFLAGS` or as Makefile variables. Architecture detection runs automatically at compile time through `arch_detect.h`.

A.1.1 Performance Optimization Options

`USE_ASM_OPT`

Enables hand-optimized, architecture-specific assembly implementations for performance-critical operations. Type: integer (0 or 1). Default: 0.

Affected routines include:

- `vm_push` / `vm_pop` — data stack operations
- `vm_rpush` / `vm_rpop` — return stack operations
- `vm_add_check_overflow` / `vm_sub_check_overflow` — arithmetic with overflow detection
- `vm_mul` / `vm_div` — multiply and divide
- `vm_strcmp_asm` / `vm_memcpy_asm` — string and memory operations
- `vm_min_asm` / `vm_max_asm` — conditional-move optimizations
- `vm_abs_asm` — absolute value (ARM64)
- `vm_clz` / `vm_ctz` — count leading/trailing zeros (ARM64)
- `vm_prefetch` / `vm_prefetch_stream` — cache prefetching (ARM64)

Typical performance impact: ≈20–25% on x86_64; ≈15–20% on ARM64. Enabled automatically by the `fastest`, `fast`, `turbo`, and `pgo` targets.

`USE_DIRECT_THREADING`

Switches the inner interpreter from indirect threading (the FORTH-79 default) to direct threading using GCC computed-goto (`&&label` / `goto *ptr`). In direct-threaded mode each word's code field points directly to native code, eliminating one indirection level per dispatch. Type: integer (0 or 1). Default: 0.

Requirements: GCC or Clang; must be combined with `USE_ASM_OPT=1`. Full support is provided for x86_64 (`vm_inner_interp_asm.h`) and ARM64 (`vm_inner_interp_arm64.h`).

Typical performance impact: ≈30–40% over indirect threading when combined with `USE_ASM_OPT=1`. Enabled by `fastest`, `fast`, and `pgo` targets.

`NDEBUG`

Standard C99 macro. When defined, all `assert()` calls compile to nothing, and StarForth additionally suppresses debug logging in hot paths. Default: undefined (assertions active). Typical impact: ≈5–10% speedup.

`STARFORTH_PERFORMANCE`

Experimental flag that trades safety for throughput. Reduces stack-bounds checking in hot paths (DUP, SWAP, etc.) and applies `UNLIKELY()` branch hints, assuming well-formed Forth input.

Caution: stack overflows may go undetected. Use only on code that has been exhaustively tested. Typical additional gain: ≈5–8% over standard `-O3`. Relevant sources: `src/word_source/stack_words.c:5` (DUP), `src/word_source/stack_words.c:117` (SWAP).

A.1.2 Architecture Options

Architecture detection is automatic via `arch_detect.h`; manual overrides are available for cross-compilation scenarios.

`ARCH_X86_64`

Targets x86-64 (AMD64 / Intel 64). Auto-detected from compiler predefined macros (`__x86_64__`, `_M_X64`, `__amd64__`). Enables x86_64 assembly optimizations, SSE4.2 string instructions, and the x86_64 direct-threading header.

`ARCH_ARM64`

Targets AArch64 (ARMv8-A). Auto-detected from `__aarch64__`, `_M_ARM64`, and `__arm64__`. Enables NEON SIMD instructions, ARM64-specific instructions (CLZ, CTZ, RBIT, REV), ARM64 assembly optimizations, and ARM64 direct threading.

Native ARM64 build: `make rpi4`. Cross-compile from x86_64: `make rpi4-cross`.

`ARCH_NAME`

Human-readable string set automatically: `"x86_64"`, `"ARM64"`, or `"Unknown"`. Displayed by `make help`.

On an unrecognized architecture the build emits a compiler warning and falls back to pure-C implementations; no assembly optimizations are loaded.

A.1.3 Platform / Target Options

`STARFORTH_MINIMAL`

Enables minimal/freestanding build suitable for bare-metal or microkernel environments. When defined: ANSI color codes are suppressed in logging; I/O is reduced to minimal output; the build links with `-nostdlib -ffreestanding`. Enabled by `make minimal` and `make l4re`.

`L4RE_TARGET`

Targets the L4Re microkernel operating system. Enables the L4Re block-storage backend, ROMFS file access in place of POSIX, and implies `STARFORTH_MINIMAL=1`. Status: partially implemented (ROMFS stub present). See `src/word_source/starforth_words.c:225` and `include/blkio_fa`

A.1.4 Debugging and Profiling Options

`DEBUG`

Debug builds compile with `-O0 -g -DDEBUG`: no optimization, full debugging symbols, all assertions enabled, verbose logging throughout. Enabled by `make debug`.

`PROFILE_ENABLED`

Activates StarForth's built-in profiler. Type: integer (0 or 1). Default: 0. The profiler tracks per-word execution counts, call-graph relationships (caller/callee), stack operation counts, and total instruction counts. Overhead: ≈10–15%.

```
./build/starforth --profile 2       # enable at depth 2
./build/starforth --profile-report # print results
```

See `include/profiler.h`.

`STRICT_PTR`

Controls the Forth address–to–C pointer mapping. Type: integer (0 or 1). Default: 1.

- `STRICT_PTR=1` — Forth addresses are real pointers (`cell_t = uintptr_t`).
- `STRICT_PTR=0` — Forth addresses are indices/offsets (segmented memory models).

Disable only for benchmarking or non-hosted targets. See `src/vm_api.c:202` and `Makefile:37`.

A.1.5 Block System Options

- `BLKCFG_DEFAULT_FBS` — default Forth block size in bytes (default: 1024). See `include/blkcfg.h`.
- `BLKCFG_PATH_MAX` — maximum path length for block-storage files (default: 4096).
- `BLK_FORTH_SYS_RESERVED` — number of system-reserved RAM blocks, hidden from user access (default: 32, blocks 0–31).
- `BLK_DISK_SYS_RESERVED` — number of system-reserved disk blocks (default: 32, PBN 1024–1055).

A.1.6 Memory Configuration Options

- `DATA_STACK_SIZE` — data stack depth in cells (default: 256). See `include/vm.h`.
- `RETURN_STACK_SIZE` — return stack depth in cells (default: 256).
- `STARFORTH_STATE_BYTES` — total state-buffer size for minimal builds (default: 8192). See `src/main.c:40-42`.
- `SF_FC_BUCKETS` — number of forget-chain hash buckets; defined in `vm.h`.

A.1.7 Feature Flags

`STARFORTH_ANSI`

Enables ANSI escape sequences for terminal control in the block editor (LIST, EDIT): screen clear (`\x1b[2J`), cursor positioning, and colored line numbers. See `src/word_source/editor_words.c:74,188,20`

`VM_HAS_CURRENT_ENTRY`

Enables the `vm->current_entry` field, which tracks the currently-compiling word. Required for the SEE decompiler, recursive word definitions, and dictionary introspection. Default: defined (enabled).

A.1.8 Build System Variables

- `CC` — C compiler. Default: `gcc`. Also accepts `clang` and `aarch64-linux-gnu-gcc`.
- `CFLAGS` — compiler flags. Extends Makefile `BASE_CFLAGS`.
- `LDFLAGS` — linker flags. Default includes `-Wl,-gc-sections -s -flto=auto -fuse-linker-plugin -static`.
- `MINIMAL` — integer (0 or 1). Setting `MINIMAL=1` enables `STARFORTH_MINIMAL` and adds `-nostdlib -ffreestanding`.
- `ASM` — integer (0 or 1). When set to 1, the build emits `.s` assembly listings alongside `.o` object files.

A.1.9 Build Configuration Matrix

Table A.1: Common build configurations

Target	USE_ASM_OPT	USE_DIRECT_THREADING	NDEBUG	Opt level	Use case
`debug`	0	0	No	`-O0 -g`	Developme
`all`	1	0	No	`-O2`	Default bu
`turbo`	1	0	Yes	`-O3 -flto`	Fast, no di
`fast`	1	1	Yes	`-O3`	Fast, reada
`fastest`	1	1	Yes	`-O3 -flto`	Maximum
`pgo`	1	1	Yes	`-O3 -fprofile-use`	Absolute fa
`minimal`	0	0	No	`-O2`	Embedded
`l4re`	0	0	No	`-O2`	L4Re micr
`profile`	0	0	No	`-O1 -g`	Profiling

A.1.10 Optimization Level Guidelines

- **`-O0 -g` (development)** — fastest compile, slowest runtime (≈10× below optimized); full symbols, all assertions. Use for initial development and crash debugging.
- **`-O2` (balanced)** — moderate compile overhead; 3–4× runtime gain over `-O0`. Default production-test builds.
- **`-O3` (high performance)** — aggressive inlining and vectorization; 5–6× over `-O0`. Production releases.

- `-O3 -flto -march=native -fprofile-use` **(maximum)** — multi-stage build; 7–8× over `-O0`. Release and benchmark artifacts. Use `make pgo`.

A.1.11 Usage Examples

Maximum performance native build

```
make clean && make pgo
```

Produces a PGO-optimized binary with assembly optimizations, direct threading, and link-time optimization.

Debug build with profiler

```
make clean && make profile
./build/starforth --profile 3 --profile-report
```

Cross-compile for Raspberry Pi 4

```
make rpi4-cross
scp build/starforth pi@raspberrypi.local:~/
```

Produces a statically linked ARM64 binary optimized for Cortex-A72.

Minimal embedded build

```
make minimal
```

Produces a freestanding binary with no libc dependencies.

AMD Zen 3 custom build

```
make clean
make CFLAGS="$(BASE_CFLAGS) -O3 -march=znver3 \
     -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DNDEBUG" \
     LDFLAGS="-flto -s"
```

Generate assembly listings

```
make asm
less build/stack_management.s
```

A.1.12 Compiler Support

- **GCC** — recommended. Minimum GCC 7.0; GCC 11.0+ preferred for improved PGO and ARM64 code generation.
- **Clang** — minimum Clang 10.0; computed-goto and all assembly optimizations supported.
- **ARM64 cross-compilation from x86_64:**

```
sudo apt-get install gcc-aarch64-linux-gnu
make CC=aarch64-linux-gnu-gcc rpi4-cross
```

Static linking (`-static`) is recommended for cross-compiled binaries.

A.1.13 Performance Tuning Checklist

For maximum throughput:

- Use `make pgo` (profile-guided optimization).
- Enable `USE_ASM_OPT=1` and `USE_DIRECT_THREADING=1`.
- Define `NDEBUG`.
- Pass `-O3 -flto -march=native`.
- Add `-fno-plt -fno-semantic-interposition` to reduce indirection.
- Add `-funroll-loops -finline-functions` for code expansion.
- Link statically (`-static`) to eliminate PLT overhead.
- Strip symbols (`-s`) to reduce binary size.

Expected result: ≈7–8× faster than a debug build; ≈1.2–1.5× faster than a plain `-O3` build.

A.1.14 Troubleshooting

"Unknown architecture" warning The build falls back automatically to pure-C implementations. Assembly optimizations are disabled. To add support for a new architecture, extend `arch_detect.h`.

Direct threading shows no performance gain Verify that both `USE_DIRECT_THREADING=1` and `USE_ASM_OPT=1` are set, that the target is x86_64 or ARM64, and that the compiler is GCC or Clang. At runtime, `WORDS-INFO` reports whether direct threading is active.

PGO coverage mismatch warnings Run `make clean && make pgo` to force a full clean two-stage rebuild. Stale coverage data from a prior instrumentation pass causes these warnings.

Appendix B

HOLA Analytics Protocol

B.1 The HOLA Shared-Memory Protocol (Phase 1)

Phase 1 establishes the Host Observation & Logistics Adapter (HOLA) as the stable contract between the StarForth VM and external analyzers. The runtime exposes a fixed-size analytics heap — 10 MiB by default — implemented in `physics_runtime.c`. This section specifies the shared-memory schema and command flow that Phase 1 tooling expects.

B.1.1 Memory Layout

The analytics heap divides into four regions. The header carries single-producer/single-consumer metadata identified by `HOLA_SHARED_MAGIC`; the event ring is a lock-free buffer of physics events; the summary region holds aggregated statistics and Bayesian posteriors; and the scratch region is reserved for governance-approved features.

Region	Offset	Size	Notes
Header	`0`	`sizeof(header_t)`	Producer/consumer metadata
Event ring	`ring_offset`	`ring_bytes`	Lock-free event ring
Summary	`summary_offset`	`summary_bytes`	Stats & posteriors
Scratch	`scratch_offset`	`scratch_bytes`	Reserved

Table B.1: HOLA analytics heap regions.

The header mirrors the C structure in `include/physics_runtime.h` and must remain packed and aligned exactly as defined. All multi-byte fields use the VM's native endianness. Consumers must validate `magic`, `version_major`, and `version_minor` before touching the heap.

B.1.2 Event Records

Each ring entry begins with a fixed header:

```
struct physics_analytics_event_header {
    uint32_t channel;         // logical stream id
    uint16_t payload_bytes;   // raw payload size
    uint16_t reserved;        // alignment padding (zero)
    uint64_t timestamp_ns;    // producer timestamp (sf_monotonic_ns)
};
```

Payloads are padded to eight bytes. The producer updates `produce_seq` atomically after writing each record; consumers advance `read_offset` and bump `consume_seq` once an event is fully processed. Overflow is signalled by incrementing `dropped_events` while leaving the ring untouched, and consumers must treat the next readable event as the resynchronization point.

B.1.3 Channels

Channel	Purpose
`0x00000001`	Word execution samples (entropy, latency, dictionary id)
`0x00000002`	Host snapshot deltas (`physics_host_snapshot_t`)
`0x0000FF00`–`0x0000FFFF`	Governance-reserved control/alert events

Table B.2: Recommended HOLA channels.

Channel `0x00000002` carries a binary `physics_host_snapshot_t` in native endianness. Phase 1 adds Pressure Stall Information averages (`avg10/60/300` scaled by 1000), `/proc/stat` totals (`cpu_total_jiffies`, `cpu_idle_jiffies`) for consumer-side delta calculation, cgroup v2 readings (`cgroup_cpu_usage_us` from `cpu.stat`, `cgroup_memory_current_bytes` from `memory.current`, each zero when unavailable), and a `flags` bitmask (`PHYSICS_HOST_FLAG_*`) indicating which groups were populated. Consumers should honor the flag bits to distinguish kernels that expose PSI, cgroup v2, or `/proc/stat`.

B.1.4 Command Protocol

Commands travel through the summary region; its first 256 bytes form a simple request/response mailbox:

```
struct hola_command {
    uint32_t opcode;       // see table
    uint32_t arg0;         // optional argument / status
    uint64_t arg1;         // optional payload offset from heap base
};
```

Opcode	Direction	Semantics
`HOLA_NOP`	Either	No-op; producer clears when idle
`HOLA_REQUEST_SNAPSHOT`	Consumer → VM	Publish snapshot on channel `0x2`
`HOLA_RESET_RING`	Consumer → VM	VM zeroes cursors when safe
`HOLA_STATUS_OK`	VM → Consumer	Command completed
`HOLA_STATUS_BUSY`	VM → Consumer	Deferred; retry next tick
`HOLA_STATUS_UNSUPPORTED`	VM → Consumer	Opcode not understood

Table B.3: HOLA command opcodes.

The VM polls the mailbox at the end of each observation-window hop and writes responses back into the same structure before the next poll. Phase 1 implements only `HOLA_REQUEST_SNAPSHOT`; the remaining opcodes are reserved.

B.1.5 Synchronization Rules

- **Single producer**: the VM is the sole writer to the ring; multiple producers must funnel through the runtime shim.
- **Single consumer**: Phase 1 assumes exactly one analyzer; multi-consumer support will require an indirection table in the summary region.
- **Memory ordering**: producers write payload, then header, then bump `produce_seq`; consumers read `produce_seq` before pulling events and bump `consume_seq` after advancing `read_offset`.

- **Failure semantics**: when `dropped_events` increases, the consumer treats the last coherent boundary as unknown and resynchronizes from the first complete record after the drop.

B.1.6 Artefacts

The runtime implementation lives in `include/physics_runtime.h` and `src/physics_runtime.c` (`physics_runtime_init`, `physics_host_snapshot`, `physics_analytics_publish_event`). The formal model linkage is in `Physics_StateMachine.thy` and `Physics_Observation.thy`, which capture the observation-window semantics this protocol assumes. Future revisions will document multi-node governance flows and captable wiring for L4Re tasks once Phase 2 moves the analyzer into a dedicated microkernel service.

Appendix C

Physics Design Notes

C.1 Physics Signal Inventory

This inventory enumerates the telemetry the physics-driven scheduler can sense today, drawn from two sources: the L4Re microkernel (through the StarshipOS loader stack) and the in-process StarForth VM. It defines the input stage of the scheduler. The guiding metaphor is an operational amplifier: the positive input ties to real host signals, the negative input ties to VM-internal counters, and a Bayesian inference loop stabilizes the gain. The metaphor is a modeling device for the control architecture, not a literal circuit.

C.1.1 L4Re and Microkernel Signals

Kernel Interface Page (KIP). The KIP exposes a high-resolution monotonic clock (`l4_kip_clock_ns()`) for timing when no RTC is present, a kernel build fingerprint for correlating inference data with kernel revisions, the scheduling granularity (an upper bound on how fine-grained the physics scheduler can react), physical memory descriptors (how much RAM can be earmarked for in-RAM inference buffers), and platform/SMP information. SMP availability doubles as a noise hook: work can be scheduled on sibling cores to inject controlled jitter and keep the model honest.

Scheduler object. `l4_scheduler_info()` reports the online CPU bitmap and maximum CPU count, bounding how many physics loops run in parallel. `l4_sched_param_t` (priority and quantum) records the loader's configured timeslice as a prior per VM. The scheduler-class flags reveal whether fixed-priority or weighted-fair-queue scheduling is active, which shapes how aggressively words may be heated or cooled. `l4_sched_cpu_set()` pins the main or physics worker thread to a core subset during experiments.

Loader and environment capabilities. The loader script (`quark.lua`) wires named capabilities the runtime can tap: `vbus_storage` for block-device enumeration and driver statistics (queue depth, error counts); `ahci_chan` for completion-interrupt rates and per-port error registers; `rtc_ns` for wall-clock sync; `cons_mux` for console input cadence as a proxy for interactive heat; `L4.Env.log` for scheduler and hardware warnings that should bias temperature decay; and `L4.Env.vesa` for framebuffer refresh/blit latency.

Further kernel APIs. `l4_thread_control()` and `l4_thread_ex_regs()` expose per-thread UTCB pointers and instruction pointers for correlating VM stalls with kernel scheduling; `l4_ipc_error()` surfaces IPC failure codes that could feed entropy penalties for fault-prone words; and virtual IRQ devices can be counted to derive environmental noise injected into the VM.

C.1.2 StarForth VM Signals

Core runtime state. `DictEntry.entropy` is the per-word execution counter, the baseline for macro temperature. `DictEntry.physics` holds the tunable knobs (temperature, last-active timestamp, mass). The data and return stack pointers (`VM.dsp`, `VM.rsp`) report stack depth, whose steep excursions imply turbulent execution and feed variance estimates. Mode and state fields reveal interpret-versus-compile state and the currently executing word. The `error` and `halted` flags are binary fault signals that spike entropy decay when they flip. Log volume and severity mix are themselves telemetry: heavy warnings can cool risky words.

Profiler and instrumentation. `WordProfile` supplies per-word total time, call count, and min/avg/max latency — a direct feed for `avg_latency_ns` and Bayesian likelihoods. `MemoryProfile` supplies read/write counts and bytes as a mass-energy proxy for memory-thrashing words. The profiler counters (VM cycles, dictionary lookups, stack ops, allocations) seed default temperatures and heat capacities. `profiler_word_count()` stays available even without detailed profiling, and `vm_debug_dump_state()` provides a structured post-mortem the Bayesian tool can parse to reset priors after a crash.

Block and storage subsystem. Block-subsystem globals report working-set size, dirty-block counts, and block-allocation-map churn. `blkio_info()` exposes device geometry and the read-only bit, informing whether a cooling word should migrate to RAM or disk tiers. `blkio_read/write` return codes give immediate error feedback. Cache slots track hit/miss rate and write-back frequency to infer subsystem momentum.

C.1.3 Op-Amp Signal Flow

The signal flow follows the amplifier metaphor in five stages:

1. **Positive input (microkernel)** — real-world noise: CPU availability, I/O latency, RTC drift, IRQ storms.
2. **Negative input (VM)** — internal state: entropy, latency, stack tension, storage dirty set.
3. **Amplifier** — the Bayesian inference loop adjusts priors and updates word physics.
4. **Output** — updated `DictPhysics` structs and scheduler hints that modulate execution order and block placement.
5. **Feedback** — the observation window width is adjusted by variance (a gauge study) and the cycle repeats.

C.1.4 Messaging and IPC Considerations

The pub/sub backbone is shared-memory-first: a ring buffer with sequence counters inside a dedicated analytics heap (10 MiB by default). Producers write events, flip a counter, and continue, with no blocking semantics inside the VM. On L4Re, where IPC is synchronous, IPC serves only as a notification channel — a publisher pokes a notification thread that drains the ring and forwards to subscribers; virtual IRQs offer an alternative non-blocking wakeup. On Linux the same API is backed by condition variables or eventfd behind a common shim, keeping the VM path identical across platforms. All state stays resident in a fixed-size analytics heap with no dynamic expansion. That heap is separate from the 5 MiB VM arena (`VM_MEMORY_SIZE`), so the dictionary, stacks, and block-subsystem budgets remain untouched.

C.1.5 Host Snapshot Shim and Analytics Heap

Phase 1 ships a concrete implementation. `physics_runtime_init()` reserves the analytics heap (10 MiB default) whose layout the HOLA contract documents, publishing the header and region descriptors HOLA consumes. `physics_host_snapshot()` abstracts POSIX and L4Re scheduler probes and feeds ring-buffer events on channel `0x00000002` via `physics_analytics_publish_event()`. The heap header, event records, and mailbox schema live in `include/physics_runtime.h`; the runtime lives in `src/physics_runtime.c`. The interface is deliberately ABI-stable so governance tooling can mirror it without pulling in C sources. On the POSIX path, Phase 1 additionally captures Linux PSI values mapped into `psi_*_avg{10,60,300}_milli`, `/proc/stat` total and idle jiffies, and cgroup v2 CPU and memory usage where available; flag bits advertise which sources were populated.

C.1.6 Conventions and Constraints

Several conventions govern the inference loop. The observation window combines a time-based heartbeat with event-count triggers: events act as "excitement" that boosts entropy, while publish and decay operations cool at roughly half that rate (tunable). All computation uses 64-bit fixed-point integers. Physics snapshots can optionally be persisted into FORTH block storage and reloaded during `(INIT)` to simulate a warm boot or replay a training sequence. Descriptor inheritance flows from module, vocabulary, and VM defaults down to individual words, so sensible priors seed at multiple levels; every tier exposes the same attribute schema — temperature, latency, mass, state flags, ACL hints, pub/sub mask, and pinned flag. VM-level rollups mirror per-word metrics and define operating bands (`COLD`, `WARM`, `HOT`, `CRITICAL`) that governance and the scheduler shim can respond to. Isabelle captures the formal state machine, invariants, and IPC-handshake proofs; HOLA defines the shared-memory layout and control protocol consumed by both the VM and external analyzers.

A primitive seed table (`physics_metadata_apply_seed()`) installs initial priors for high-impact primitives — control flow, I/O, block subsystem, save-system — so temperature and latency estimates do not start at absolute zero; governance tooling can extend or override the table once the Bayesian loop is in place.

C.1.7 Immediate Research Tasks

1. Prototype a thin KIP/scheduler shim exposing the listed signals to userland C with no filesystem: on L4Re via `l4re_kip()`, on POSIX via a stub that feeds monotonic time and scheduler defaults.

2. Inventory the IO-server (vbus) protocol to pull queue-depth and error counters for AHCI, NVMe, and virtio backends.

3. Define the shared-memory layout between the VM and the Bayesian analyzer, defaulting to a 10 MiB heap (header, ∼6 MiB event ring, ∼3 MiB summary/scratch, padding), honoring the platform split between POSIX and L4Re messaging.

4. Extend the profiler to snapshot `MemoryProfile` deltas without enabling full verbose mode, keeping overhead low.

5. Derive initial priors for key primitives (control, I/O, block) from handcrafted knowledge plus the loader's priority and quantum configuration.